Lab 3

Xử lý các tình huống thực tế phức tạp

Hình thức nộp: Đường dẫn repository GitHub (public) — làm tiếp trên repo của Bài 1–2

---

1. Mục tiêu

Sau bài tập này, sinh viên có thể:

- Nhận diện nguyên nhân test chập chờn (flaky) và chữa bằng Explicit Wait.
- Xử lý alert, iframe và các tương tác nâng cao (dropdown).
- Viết Assert kiểm chứng đúng bản chất nghiệp vụ, không chỉ đếm số lượng.
- Tự động chụp ảnh màn hình khi test thất bại để truy vết.
- Dùng AI để chẩn đoán flaky test và tự thẩm định gợi ý.

2. Yêu cầu

Phần A — Chứng minh vấn đề đồng bộ (25%)

- Dùng trang `the-internet.herokuapp.com/dynamic_loading/2`: bấm nút Start, chờ phần tử `#finish` (chứa chữ "Hello World!") được JavaScript tạo ra sau vài giây.
- Bước 1 — viết test tìm `#finish` NGAY sau khi bấm Start (không Wait); chụp ảnh test **RỚT** kèm thông báo lỗi `NoSuchElementException`.
- Bước 2 — thêm Explicit Wait, chụp ảnh test **PASS**.
- Viết 2–3 câu giải thích vì sao cùng một code mà kết quả lại khác nhau.
- Yêu cầu: dùng `WebDriverWait` + `Until(...)` — **KHÔNG** được dùng `Thread.Sleep`.
- Lưu ý quan trọng: **không dùng** `performance_glitch_user` cho phần này. Tài khoản đó chỉ làm trang tải CHẬM, mà việc chờ tải trang thì Selenium đã tự làm giùm (`pageLoadStrategy = normal`) — nên test sẽ chậm nhưng vẫn PASS, không chứng minh được điều gì.

Phần B — Dropdown & Assert đúng nghiệp vụ (25%)

- Sau khi đăng nhập, dùng `SelectElement` chọn sắp xếp **Price (low to high)** (dropdown `product_sort_container`).
- Lấy danh sách 6 giá bằng `FindElements`; bỏ ký tự $ và chuyển sang kiểu số.
- Assert danh sách giá **THỰC SỰ tăng dần** — không chấp nhận chỉ Assert số lượng = 6.

- Gợi ý: so sánh danh sách vừa lấy với chính nó sau khi đã sắp xếp.

Phần C — Alert & iframe (20%)

- Trên `the-internet.herokuapp.com/javascript_alerts`: xử lý đủ **3 loại** — Alert (`Accept`), Confirm (`Dismiss`), Prompt (`SendKeys` + `Accept`).
- Mỗi ca đều phải Assert nội dung hiển thị ở `#result`.
- Trên `/iframe`: dùng `SwitchTo().Frame(...)` gõ một đoạn chữ vào trình soạn thảo, sau đó gọi `DefaultContent()`.

Phần D — Chụp ảnh khi test thất bại (15%)

- Thêm cơ chế chụp ảnh vào `[TearDown]`, CHỈ chụp khi `Outcome.Status == TestStatus.Failed`.
- Đặt tên file ảnh theo tên ca test; đính kèm bằng `TestContext.AddTestAttachment(path)`.
- Cố ý làm 1 test rớt để sinh ảnh; nộp kèm ảnh đó làm minh chứng.

Phần E — AI chẩn đoán flaky test (15%)

- Chọn 1 test chập chờn (có thể dùng lại tình huống Phần A), mô tả cho AI đầy đủ: **code, lỗi nguyên văn, tần suất rớt, điều kiện chạy**.
- Ghi lại prompt đã dùng và các nguyên nhân AI đề xuất.
- Tự kiểm chứng từng gợi ý; viết nhận xét (3–5 câu): gợi ý nào đúng, gợi ý nào sai/không liên quan?

3. Hướng dẫn thực hiện sơ bộ

1. **Thêm gói Selenium.Support**: chạy `dotnet add package Selenium.Support` để có `WebDriverWait` và `SelectElement`.
2. **Khai báo wait ở cấp class**, khởi tạo trong `[SetUp]`: `wait = new WebDriverWait(driver, TimeSpan.FromSeconds(10));`
3. **Dùng cú pháp lambda**: `wait.Until(d => d.FindElement(By.Id("finish")));`. Lưu ý: từ Selenium 4, `ExpectedConditions` đã tách sang gói cộng đồng riêng — cú pháp lambda không cần cài thêm gì.
4. **Hiểu Selenium chờ giùm gì**: WebDriver tự chờ trang tải xong khi CHUYỂN trang, nhưng KHÔNG chờ phần tử do JavaScript tạo ra sau đó. Phần A khai thác đúng khoảng trống này.
5. **KHÔNG trộn Implicit và Explicit Wait** — chọn Explicit Wait và bỏ hẳn Implicit để thời gian chờ không cộng dồn.
6. **Với alert**: `driver.SwitchTo().Alert()`. Nhớ Assert `alert.Text` TRƯỚC khi Accept/Dismiss.

7. **Với iframe**: vào bằng `SwitchTo().Frame("mce_0_ifr")`, ra bằng `SwitchTo().DefaultContent()`. Vào thì phải ra — như đóng ngoặc.
8. **Chụp ảnh**: ép kiểu `((ITakesScreenshot)driver).GetScreenshot()`; cần `using NUnit.Framework.Interfaces;` cho `TestStatus`.
9. **Xác nhận đã hết flaky**: chạy lại 10–20 lần đều xanh mới kết luận. Xanh 1 lần không chứng minh được gì.
10. **Commit & push** theo chuẩn Conventional Commits như Bài 2 (ví dụ: `fix: them explicit wait cho test dang nhap`).

4. Sản phẩm nộp

- Link repository GitHub (**public**) — repo của Bài 1–2.
- Các file test cho Phần A, B, C; `[TearDown]` có cơ chế chụp ảnh (Phần D).
- Thư mục `screenshots/`: ảnh test RỚT & PASS (Phần A), ảnh minh chứng chụp khi Failed (Phần D).
- File `README.md`: giải thích Phần A và phần ghi nhận dùng AI (Phần E).

5. Tiêu chí chấm

Tiêu chí	Điểm	Ghi chú
Phần A — Chứng minh RỚT/PASS trên /dynamic_loading/2	**25%**	2 ảnh + giải thích
Phần B — SelectElement + Assert giá thực sự tăng dần	**25%**	Không chỉ đếm số lượng
Phần C — Xử lý 3 loại alert & gõ chữ vào iframe	**20%**	Có Assert #result
Phần D — Chụp ảnh tự động khi test Failed	**15%**	Chỉ chụp khi Failed
Phần E — AI chẩn đoán flaky + tự thẩm định	**15%**	Prompt + nhận xét riêng
Tổng cộng	**100%**	

Lưu ý: bài này KHÔNG chấp nhận `Thread.Sleep` để chờ trang. Mọi cơ chế chờ phải dùng Explicit Wait — đây là tiêu chí chấm bắt buộc của Phần A.